

GEMM from scratch

Juan Esteban Garcia Pulgarin

April 24, 2026

Contents

1	Hardware profile	3
2	FLOPs and throughput	4
3	Step-by-step analysis	6
3.1	Sequential CPU Matrix Multiply in C	6
3.2	Naive CUDA kernel	7
3.2.1	Arithmetic Intensity	8
3.3	Tiled GEMM with shared memory	8
3.3.1	Arithmetic Intensity	9
3.4	Bank conflict elimination	10
3.5	Register tiling	12
3.5.1	Arithmetic Intensity	13
3.6	Vectorized loads + read-only cache	14
4	Global performance analysis	15
4.1	Throughput at $N = 1024$	15
4.2	Throughput scaling with matrix size	16
4.3	Execution time scaling	17
4.4	Speedup relative to the naive kernel	18
4.5	Arithmetic intensity at $N = 1024$	18
4.6	Hardware efficiency	19
4.7	Roofline interpretation	20
4.8	Summary	21
5	Conclusion	22

List of Figures

1.1	Hardware efficiency at $N=1024$	3
2.1	Performance comparison of GEMM implementations across increasing matrix sizes ($N = 512, 1024, 2048, 4096$), showing the evolution of elapsed time and achieved throughput (TFLOP/s) as problem size scales.	4
3.1	Throughput and elapsed time in sequential CPU multiply	6
3.2	Throughput and elapsed time in Naive CUDA kernel	7
3.3	Throughput and elapsed time for tiled GEMM with shared memory	9
3.4	Comparison between tiled and padded GEMM implementations for $TILE=16$ and $TILE=32$. The results show that padding shared memory does not improve performance in this kernel and instead introduces a slight slowdown due to increased shared memory usage and reduced occupancy.	11
3.5	Throughput and elapsed time for register-tiled GEMM with $TILE = 64$ and $TW = 4$	12
3.6	Throughput and elapsed time for vectorized loads with read-only cache	14
4.1	TFLOP/s comparison across implementations at $N = 1024$	15
4.2	TFLOP/s versus matrix size for all implementations.	16
4.3	Execution time versus matrix size for all implementations.	17
4.4	Speedup of each implementation relative to the naive GPU kernel at $N = 1024$	18
4.5	Arithmetic intensity of each implementation at $N = 1024$	19
4.6	Hardware efficiency of each implementation relative to the T4 FP32 peak.	20
4.7	Roofline analysis at $N = 1024$	21

Chapter 1

Hardware profile

According to the specifications [1], the GPU used is a **NVIDIA Tesla T4**. Its peak performance with FP32 is **8.141 TFLOP/s**. I used an approximate value of **8.1**. Its memory bandwidth is **320.0 GB/s**.

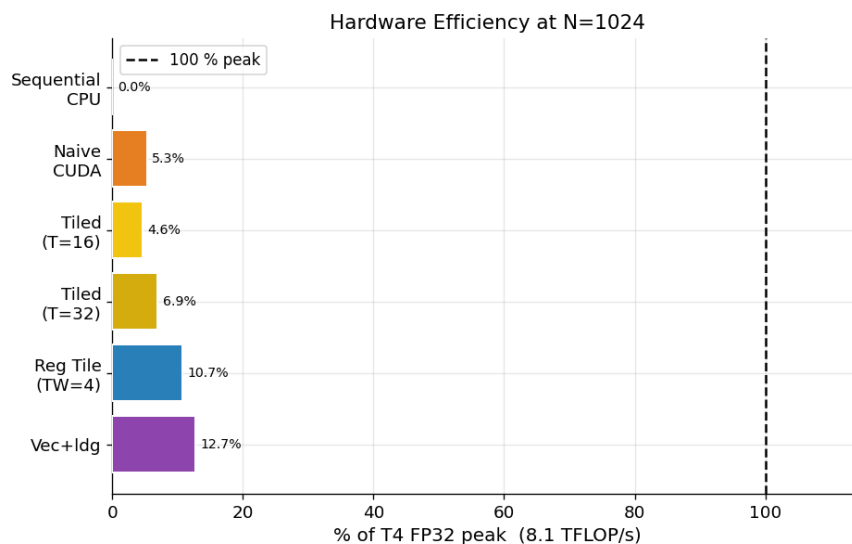
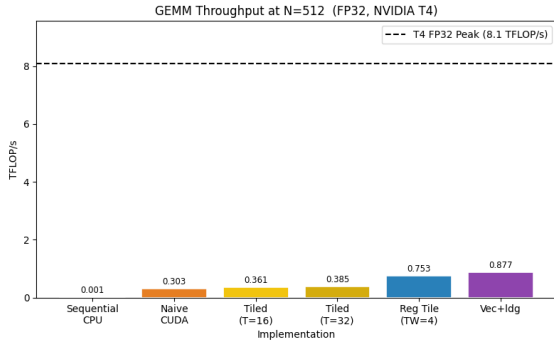


Figure 1.1: Hardware efficiency at N=1024

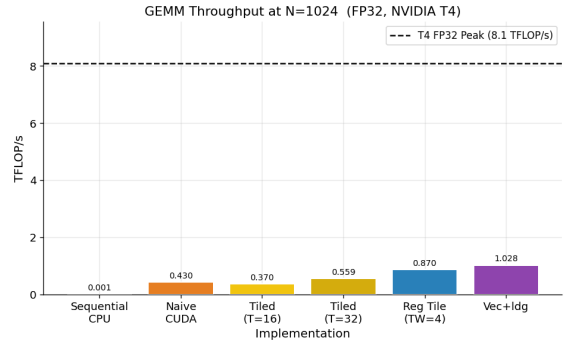
In Figure 1.1 we can see that the best percentage peak is the **Vec+ldg** implementation. That is exactly right, just as expected from that optimization.

Chapter 2

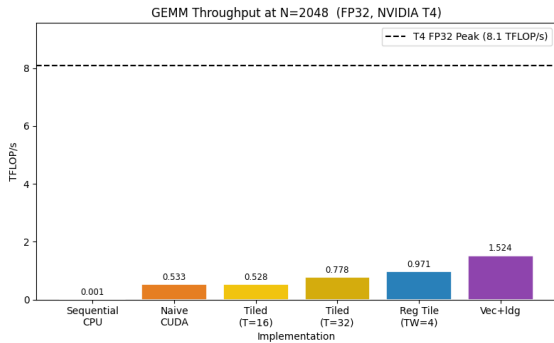
FLOPs and throughput



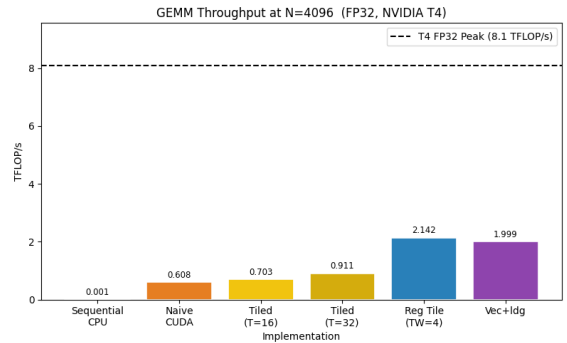
(a) TFLOP/s at N=512



(b) TFLOP/s at N=1024



(c) TFLOP/s at N=2048



(d) TFLOP/s at N=4096

Figure 2.1: Performance comparison of GEMM implementations across increasing matrix sizes ($N = 512, 1024, 2048, 4096$), showing the evolution of elapsed time and achieved throughput (TFLOP/s) as problem size scales.

To compute the matrix product $C = A \cdot B$, where $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$: Each entry of C is a dot product of length K , requiring $2K$ operations. Since there are $M \times N$ entries, the total computational cost is $2MNK$.

Given this information, we must keep in mind that the time is measured in milliseconds,

so we need to convert the units to seconds. We must also keep in mind that the units must be measured in terabytes, so we need to perform that conversion as well.

All of that give us the next equation to measure TFLOP/s with $flops = 2N^3$:

$$TFLOPs/s = \frac{flops}{\frac{\text{time_ms}}{1000}} * 10^{12}$$

So all we need to do to calculate the TFLOPs/s is substitute the core execution time and the size of the matrix N we are measuring.

TFLOP/s is low for small N because the computation is too small to fully amortize fixed costs such as kernel launch overhead, memory latency, and synchronization. In addition, the GPU cannot be fully occupied, so many execution units remain underutilized. As N increases, the total number of floating-point operations grows as N^3 , while these overheads grow much more slowly. The result is better hardware utilization, higher effective arithmetic intensity, and therefore higher measured TFLOP/s.

As shown in Figure 3.4, the achieved TFLOP/s remains relatively low and similar across implementations for smaller matrix sizes, indicating that performance is dominated by overheads and limited GPU utilization. In this regime, the problem size is insufficient to fully occupy the hardware or amortize kernel launch and memory access costs.

However, as N increases, a clear divergence appears: more optimized implementations, particularly register tiling and vectorized loads with ldg, achieve significantly higher TFLOP/s. This is because larger matrices increase the total computational workload $O(N^3)$, improving arithmetic intensity and enabling better reuse of data in shared memory and registers. As a result, the GPU becomes compute-bound rather than memory-bound, allowing these optimizations to fully exploit the hardware and deliver substantially higher throughput.

Chapter 3

Step-by-step analysis

3.1 Sequential CPU Matrix Multiply in C

Based on theory, we can say that the worst performance we can achieve is on the CPU, since it executes one instruction per clock cycle. With large arrays, execution time increases exponentially, and the number of bytes reused per clock cycle does not reach optimal performance.

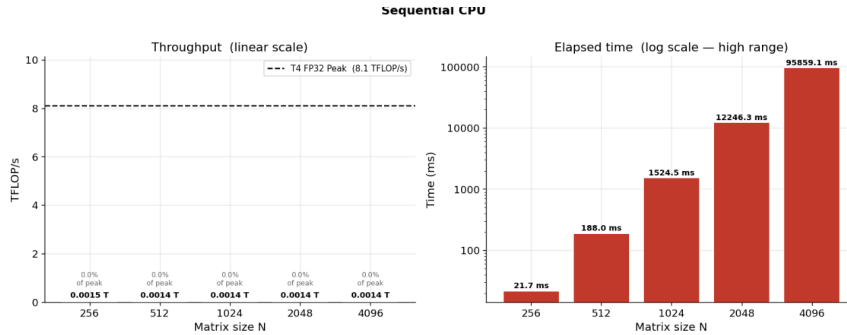


Figure 3.1: Throughput and elapsed time in sequential CPU multiply

The sequential CPU implementation Figure 3.1, serves as the correctness baseline. Since matrix multiplication performs $2N^3$ floating-point operations, the runtime grows approximately by a factor of eight whenever N doubles, which is consistent with the observed timings in the figure. For example, the execution time increases from 21.73 ms at $N=256$ to 188.05 ms at $N=512$, and continues to scale similarly for larger sizes. However, the achieved throughput remains nearly constant at about 1.4×10^{-3} TFLOP/s across all tested values of N . This indicates that increasing problem size increases the total amount of work, but does not improve the efficiency of the implementation. The reason is that the naive triple-loop code has very low data reuse: each multiplication uses one element from A and one from B , so the arithmetic intensity stays fixed at a low value. In other words, the computation is limited more by memory traffic and lack of parallelism than by raw arithmetic capability, which explains why throughput does not increase with N .

Since the arithmetic intensity is constant and low, the sequential kernel does not become

more efficient as N increases; it only spends more time doing the same inefficient pattern at a larger scale.

3.2 Naive CUDA kernel

The naive CUDA kernel is the first step that moves the computation from the CPU to the GPU. Compared with the sequential implementation, the mathematical algorithm does not change: each output element is still computed with the same triple-nested access pattern. What changes is the execution model. Instead of one CPU core iterating over the full matrix, the GPU launches many threads so that multiple output elements are computed in parallel. In this step, the host still allocates and initializes the matrices in CPU memory and transfers them to device memory before launching the kernel.

From a theoretical point of view, this optimization should improve performance because the GPU can exploit massive data parallelism. However, it does not improve arithmetic intensity, since each thread still reads one element from A and one element from B for every multiply-add, so the ratio of computation to global memory traffic remains essentially the same as in the sequential version. For that reason, the kernel is still memory-bound rather than compute-bound.

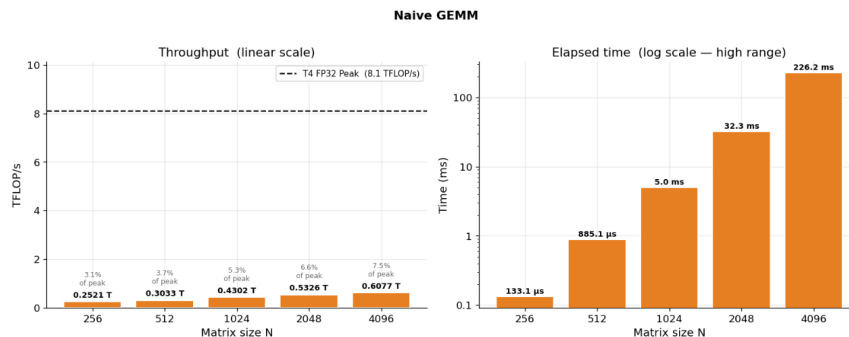


Figure 3.2: Throughput and elapsed time in Naive CUDA kernel

Even with that limitation, the execution time improves substantially compared with the sequential baseline 3.1. For example, the runtime drops from 21.73 ms to 0.133 ms at $N = 256$, from 188.05 ms to 0.885 ms at $N = 512$, from 1524.47 ms to 4.992 ms at $N = 1024$, from 12246.31 ms to 32.259 ms at $N = 2048$, and from 95859.14 ms to 226.175 ms at $N = 4096$ 3.2. These results correspond to speedups of $168.07\times$, $216.64\times$, $307.29\times$, $330.43\times$, and $434.07\times$, respectively. This matches the theoretical expectation that the GPU should outperform the CPU by a large margin due to parallel execution, although the achieved TFLOP/s remains far below peak because the kernel still performs heavy global-memory traffic and does not reuse data efficiently.

3.2.1 Arithmetic Intensity

The arithmetic intensity (AI) measures how much computation is performed per byte of data transferred from memory. It is defined as:

$$\text{Arithmetic Intensity} = \frac{\text{Total floating-point operations (FLOPs)}}{\text{Total memory traffic (bytes)}}$$

For matrix multiplication of two square matrices of size $N \times N$, the total number of floating-point operations is fixed and given by:

$$\text{FLOPs} = 2N^3$$

since each output element requires N multiply-add operations.

However, the total memory traffic depends on how data is accessed and reused. In both the sequential CPU implementation and the naive GPU kernel, there is no effective data reuse: each multiplication loads one element from A and one from B from global memory.

Therefore, the total number of memory reads is approximately $2N^3$, and the number of writes is N^2 (one per output element).

Assuming single-precision floating-point values (4 bytes per element), the total memory traffic is:

$$\text{Bytes} = (2N^3 + N^2) \cdot 4$$

Thus, the arithmetic intensity becomes:

$$\text{Arithmetic Intensity} = \frac{2N^3}{(2N^3 + N^2) \cdot 4}$$

For large N , the N^3 term dominates over N^2 , and the expression simplifies to:

$$\text{Arithmetic Intensity} \approx \frac{2N^3}{2N^3 \cdot 4} = \mathbf{0.25 \text{ FLOP/byte}}$$

This shows that both the CPU implementation and the naive GPU kernel have the same low and constant arithmetic intensity. Although the GPU version achieves higher performance due to massive parallelism, it does not improve computational efficiency, since the ratio of computation to memory traffic remains unchanged.

3.3 Tiled GEMM with shared memory

The tiled CUDA kernel improves the naive implementation by introducing data reuse through shared memory. In the naive kernel, each thread repeatedly loads elements of A and B directly from global memory for every multiply-add operation. In contrast, the tiled version partitions the matrices into $TILE \times TILE$ submatrices, cooperatively loads each tile into shared memory, and then reuses those values across multiple output computations before moving to the next tile.

From a theoretical point of view, this should improve performance because shared memory has much lower latency than global memory, and because each global-memory element is reused multiple times after being loaded. Therefore, the kernel reduces memory traffic per floating-point operation and increases arithmetic intensity. Since GEMM is a bandwidth-sensitive operation, reducing global memory accesses should lead to better throughput, especially for larger matrices where the cost of loading tiles can be amortized more effectively.

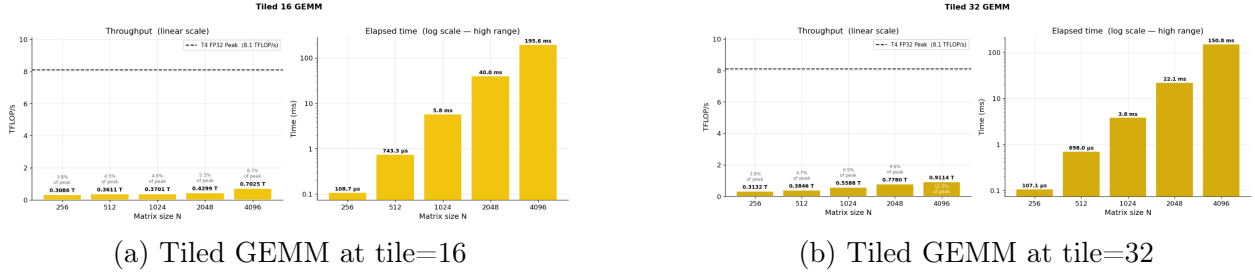


Figure 3.3: Throughput and elapsed time for tiled GEMM with shared memory

The measured results confirm part of this expectation, but not uniformly across all matrix sizes. For $TILE = 16$, the kernel reaches 0.3086 TFLOP/s at $N = 256$ and 0.7025 TFLOP/s at $N = 4096$ 3.3a, while for $TILE = 32$ it reaches 0.3132 TFLOP/s at $N = 256$ and 0.9114 TFLOP/s at $N = 4096$ 3.3b. Compared with the naive CUDA kernel, the tiled implementation becomes clearly more effective at larger sizes, where the cost of loading each tile is amortized over more reuse. However, the improvement is not dramatic for the smallest matrices, and the $TILE = 16$ version is sometimes slower than the naive kernel. This happens because shared-memory tiling introduces extra synchronization, indexing, and shared-memory access overheads, and those costs are not fully compensated when the tile size is too small.

Overall, the data show that $TILE = 32$ performs better than $TILE = 16$, which matches the theory. A larger tile increases data reuse and reduces the fraction of time spent on tile-management overheads. In addition, $TILE = 32$ aligns better with the warp size, which helps the GPU execute the kernel more efficiently.

3.3.1 Arithmetic Intensity

The arithmetic intensity (AI) measures the number of floating-point operations performed per byte of *global memory* traffic:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes transferred from global memory}}$$

For square matrix multiplication of size $N \times N$, the total number of floating-point operations is still:

$$\text{FLOPs} = 2N^3$$

What changes in the tiled version is the amount of data fetched from global memory. Each tile of A and B is loaded once into shared memory and then reused $TILE$ times by the

threads in the block. This reduces the number of global-memory reads by approximately a factor of $TILE$.

For a tile size T , the total global-memory traffic can be modeled as:

$$\text{Bytes}_A \approx \frac{N^3}{T} \cdot 4, \quad \text{Bytes}_B \approx \frac{N^3}{T} \cdot 4, \quad \text{Bytes}_C = N^2 \cdot 4$$

so the total traffic is:

$$\text{Bytes} = \left(\frac{2N^3}{T} + N^2 \right) \cdot 4$$

Thus, the arithmetic intensity becomes:

$$\text{AI}_{\text{tiled}} = \frac{2N^3}{\left(\frac{2N^3}{T} + N^2 \right) \cdot 4}$$

For large N , the N^3 term dominates, so the expression simplifies to:

$$\text{AI}_{\text{tiled}} \approx \frac{2N^3}{\frac{2N^3}{T} \cdot 4} = \frac{T}{4}$$

Therefore:

$$\text{AI}_{TILE=16} \approx \frac{16}{4} = 4 \text{ FLOP/byte}$$

$$\text{AI}_{TILE=32} \approx \frac{32}{4} = 8 \text{ FLOP/byte}$$

These values are consistent with the measured results, which approach 3.88 and 7.97 FLOP/byte, respectively. This is a major improvement over the naive kernel, whose arithmetic intensity is only about 0.25 FLOP/byte. The increase in AI explains why tiled GEMM is significantly faster: the GPU performs more computation for each byte fetched from global memory, making better use of the available compute resources.

3.4 Bank conflict elimination

In this step, the tiled GEMM kernel is modified to eliminate potential shared-memory bank conflicts by adding padding to the shared memory arrays. Specifically, the declarations are changed from:

```
float As[TILE][TILE];    float Bs[TILE][TILE];
```

to:

```
float As[TILE][TILE+1];  float Bs[TILE][TILE+1];
```

No other part of the kernel is modified: the computation, memory access pattern, and thread mapping remain identical to the tiled implementation from the previous step.

From a theoretical perspective, this optimization should improve performance by eliminating shared-memory bank conflicts. Shared memory is divided into 32 banks, and when multiple threads in a warp access different addresses that map to the same bank, accesses are serialized. For a 2D array stored in row-major order, the bank index is given by:

$$\text{bank} = (\text{row} \cdot \text{stride} + \text{col}) \mod 32$$

When $\text{stride} = \text{TILE} = 32$, accesses to a column can map all threads to the same bank, producing a 32-way conflict. By increasing the stride to $\text{TILE} + 1 = 33$, the mapping becomes:

$$\text{bank} = (\text{row} + \text{col}) \mod 32$$

which distributes accesses across all banks and should eliminate conflicts.

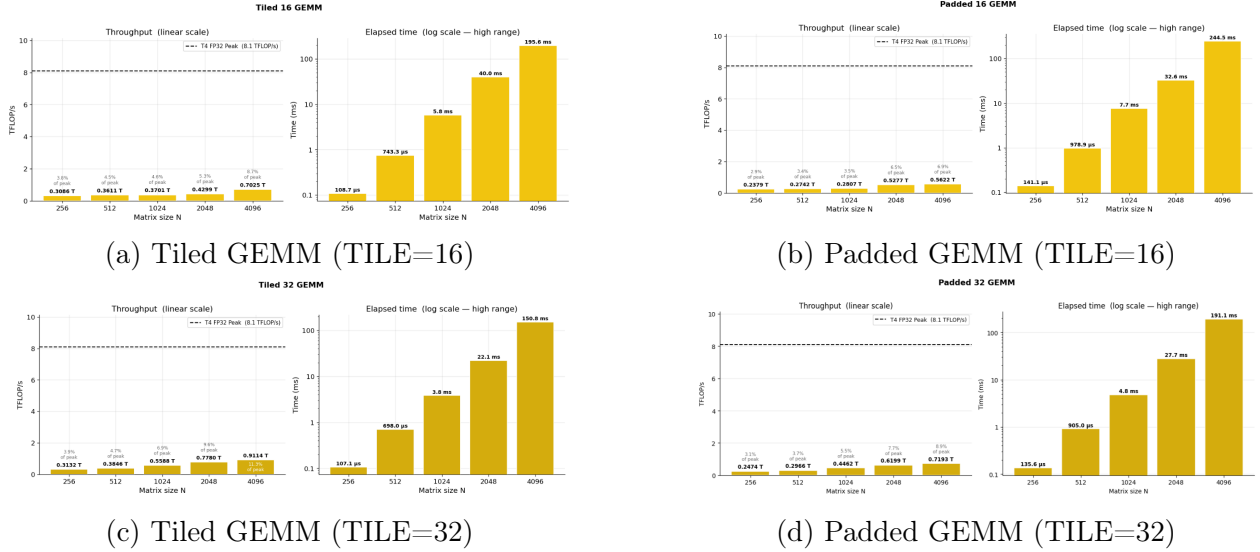


Figure 3.4: Comparison between tiled and padded GEMM implementations for $\text{TILE}=16$ and $\text{TILE}=32$. The results show that padding shared memory does not improve performance in this kernel and instead introduces a slight slowdown due to increased shared memory usage and reduced occupancy.

However, the measured results do not match this theoretical expectation. Instead of improving performance, the padded kernel is consistently slower than the original tiled version. For example, at $N = 1024$, the execution time increases from 3.843 ms (tiled, $\text{TILE} = 32$) 3.4c to 4.813 ms (padded) 3.4d, corresponding to a slowdown of approximately $1.25\times$. A similar degradation is observed for $\text{TILE} = 16$, where the padded version is about $1.32\times$ slower.

This discrepancy can be explained by analyzing the actual memory access pattern of the kernel. During the compute phase, each warp accesses shared memory in the following way:

- Access to $A_s[\text{ty}][k]$: all threads in the warp share the same ty and k , resulting in a broadcast from a single address. This does not generate bank conflicts.

- Access to $B_s[k][tx]$: k is fixed for the iteration, while tx varies from 0 to 31 across the warp. This produces accesses to consecutive memory locations:

$$\text{bank} = (k \cdot 32 + tx) \mod 32 = tx$$

which maps perfectly to all 32 banks without conflicts.

Therefore, the original tiled kernel does not suffer from bank conflicts in this access pattern. As a result, the padding does not eliminate any bottleneck, and instead introduces additional overhead. In particular, increasing the shared-memory footprint per block reduces the number of blocks that can be scheduled concurrently on each SM, lowering occupancy and increasing latency stalls.

In summary, although padding is theoretically effective for eliminating column-wise bank conflicts, it does not provide a benefit in this specific kernel because the access pattern already avoids conflicts. Consequently, the optimization results in a performance regression rather than a speedup.

3.5 Register tiling

The register-tiled kernel extends shared-memory tiling by assigning each thread a small output micro-tile instead of a single element of C . In this implementation, $TILE = 64$ and $TW = 4$, so each thread computes a 4×4 block of outputs. The block still cooperatively loads a 64×64 tile of A and B into shared memory, but inside the compute phase each thread loads TW values from the current column of A_s and TW values from the current row of B_s , then performs the full outer product in registers.

Compared with the previous tiled version, the main change is that the work is no longer limited to one output element per thread. Instead, each thread reuses the same shared-memory values to update 16 accumulators in registers before writing the results back to global memory. The theoretical reason this should help is that register reuse is faster than shared-memory reuse, and each load now contributes to more floating-point operations. In other words, the kernel increases data reuse, reduces the relative cost of memory access, and raises the amount of useful computation performed per thread.

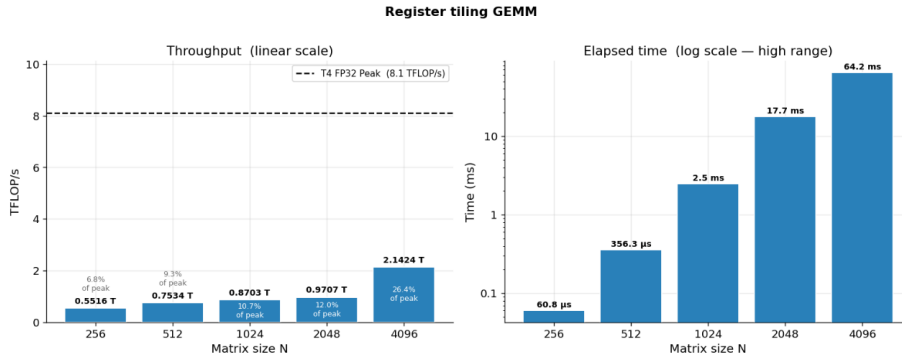


Figure 3.5: Throughput and elapsed time for register-tiled GEMM with $TILE = 64$ and $TW = 4$

The measured results confirm this trend, especially for large matrices. The kernel reaches 0.5516 TFLOP/s at $N = 256$, 0.8703 TFLOP/s at $N = 1024$, and 2.1424 TFLOP/s at $N = 4096$, while the execution time drops to 64.151 ms at the largest size 3.5. Relative to the tiled $TILE = 32$ kernel, the register-tiled version is faster by $1.75\times$ at $N = 256$, $1.96\times$ at $N = 512$, $1.56\times$ at $N = 1024$, $1.25\times$ at $N = 2048$, and $2.35\times$ at $N = 4096$. This matches the theory: the benefit becomes more visible as N grows, because the extra register work and indexing overhead are better amortized over a larger amount of computation.

3.5.1 Arithmetic Intensity

The arithmetic intensity (AI) measures the number of floating-point operations performed per byte transferred from global memory:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes transferred from global memory}}$$

For square matrix multiplication of size $N \times N$, the total floating-point work is still:

$$\text{FLOPs} = 2N^3$$

What changes in the register-tiled kernel is the amount of global memory traffic that is needed to feed that work. Since each 64×64 tile of A and B is reused across a 4×4 micro-tile per thread, the same loaded data contributes to many more FMAs before the next global load is needed. Using the same model as in the tiled case, the global traffic can be approximated as:

$$\text{Bytes} = \left(\frac{2N^3}{TILE} + N^2 \right) \cdot 4$$

with $TILE = 64$. Therefore, the arithmetic intensity becomes:

$$\text{AI}_{\text{regtile}} = \frac{2N^3}{\left(\frac{2N^3}{64} + N^2 \right) \cdot 4}$$

For large N , the N^3 term dominates, so:

$$\text{AI}_{\text{regtile}} \approx \frac{2N^3}{\frac{2N^3}{64} \cdot 4} = 16 \text{ FLOP/byte}$$

This is consistent with the measured values, which rise from 14.22 F/B at $N = 256$ to 15.88 F/B at $N = 4096$. The increase in arithmetic intensity explains the performance gain: the kernel does more useful computation for each byte fetched from global memory, which moves it closer to the compute-efficient regime of the GPU. Compared with the tiled kernel, register tiling roughly doubles the arithmetic intensity, and that is why the improvement is especially clear at larger matrix sizes.

3.6 Vectorized loads + read-only cache

This step applies two optimizations to the register-tiled kernel: vectorized memory loads using `float4` and the use of the read-only cache through the `__ldg()` intrinsic. Unlike previous steps, the computational structure of the kernel remains unchanged: each thread still computes a $TW \times TW$ output tile using shared memory and registers. The only modification is in how data is loaded from global memory into shared memory.

From a theoretical perspective, these optimizations should improve performance by making memory access more efficient. First, vectorized loads allow each thread to fetch 128 bits (4 floats) per instruction instead of 32 bits, reducing the number of load instructions issued. Second, `__ldg()` routes global memory accesses through the read-only cache, which can reduce pressure on the L1 cache and improve bandwidth utilization when data is reused across threads.

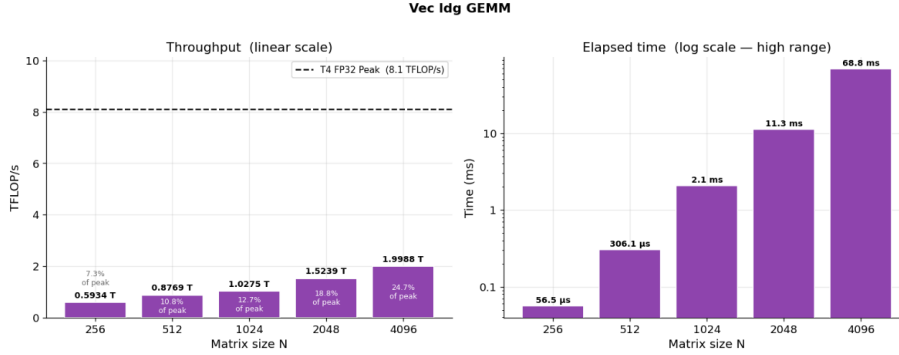


Figure 3.6: Throughput and elapsed time for vectorized loads with read-only cache

The measured results show moderate improvements compared to the register-tiled kernel, although the gains are not uniform across all matrix sizes. For example, at $N = 1024$, the performance increases from 0.8703 TFLOP/s to 1.0275 TFLOP/s, and at $N = 2048$, from 0.9707 TFLOP/s to 1.5239 TFLOP/s, corresponding to speedups of approximately $1.18\times$ and $1.57\times$, respectively. However, at $N = 4096$, the performance slightly decreases from 2.1424 TFLOP/s to 1.9988 TFLOP/s.

This behavior can be explained by the nature of the optimization. Unlike tiling or register blocking, vectorized loads and `__ldg()` do not increase arithmetic intensity or data reuse. Instead, they reduce instruction overhead and improve memory access efficiency. As a result, their impact depends strongly on whether the kernel is limited by instruction throughput or memory bandwidth.

Overall, the results partially match the theoretical expectations: the optimization improves performance when memory access efficiency is a limiting factor, but does not provide consistent gains at all scales because it does not address the fundamental bottleneck of global memory traffic.

Chapter 4

Global performance analysis

This section summarizes the behavior of all implementations and connects the main performance metrics across the optimization pipeline. Rather than analyzing each kernel in isolation, the goal here is to identify the global trends: how throughput changes with matrix size, how execution time scales, how arithmetic intensity evolves, and how close each implementation gets to the hardware peak.

4.1 Throughput at $N = 1024$

Figure 4.1 compares the TFLOP/s achieved by each implementation at $N = 1024$. The figure shows a clear upward trend across the optimization stages, with the naive CUDA kernel already providing a large gain over the sequential CPU baseline, and the best results obtained by the register-tiled and vectorized implementations. This confirms that the main source of improvement is not a change in the mathematical operation itself, but a progressive reduction in memory overhead and better use of on-chip resources.

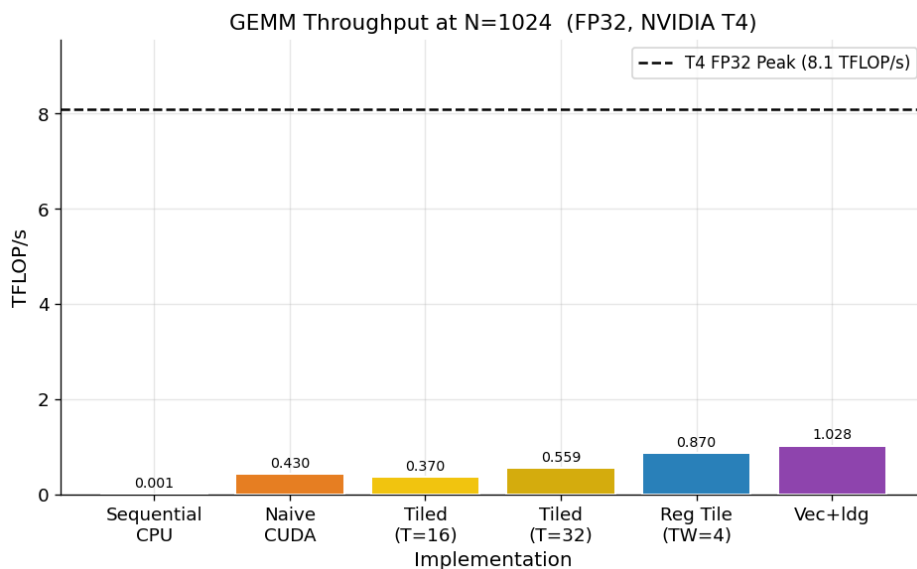


Figure 4.1: TFLOP/s comparison across implementations at $N = 1024$.

4.2 Throughput scaling with matrix size

Figure 4.2 shows TFLOP/s as a function of matrix size for all implementations. Two different behaviors are visible. First, the sequential CPU version remains essentially flat and very low, which indicates that increasing N only increases the amount of work, but does not improve efficiency. Second, the GPU implementations improve as the matrix size grows, especially for the more optimized kernels. This is because larger matrices provide more work per launch, allow better amortization of fixed overheads, and improve the opportunity for data reuse.

The naive and lightly optimized kernels stay relatively close to one another over the full range of N , which suggests that they are still constrained by memory traffic. In contrast, the register-tiled and vectorized kernels separate more clearly from the rest as N increases, showing that optimizations which improve reuse and reduce instruction overhead become more valuable at larger problem sizes.

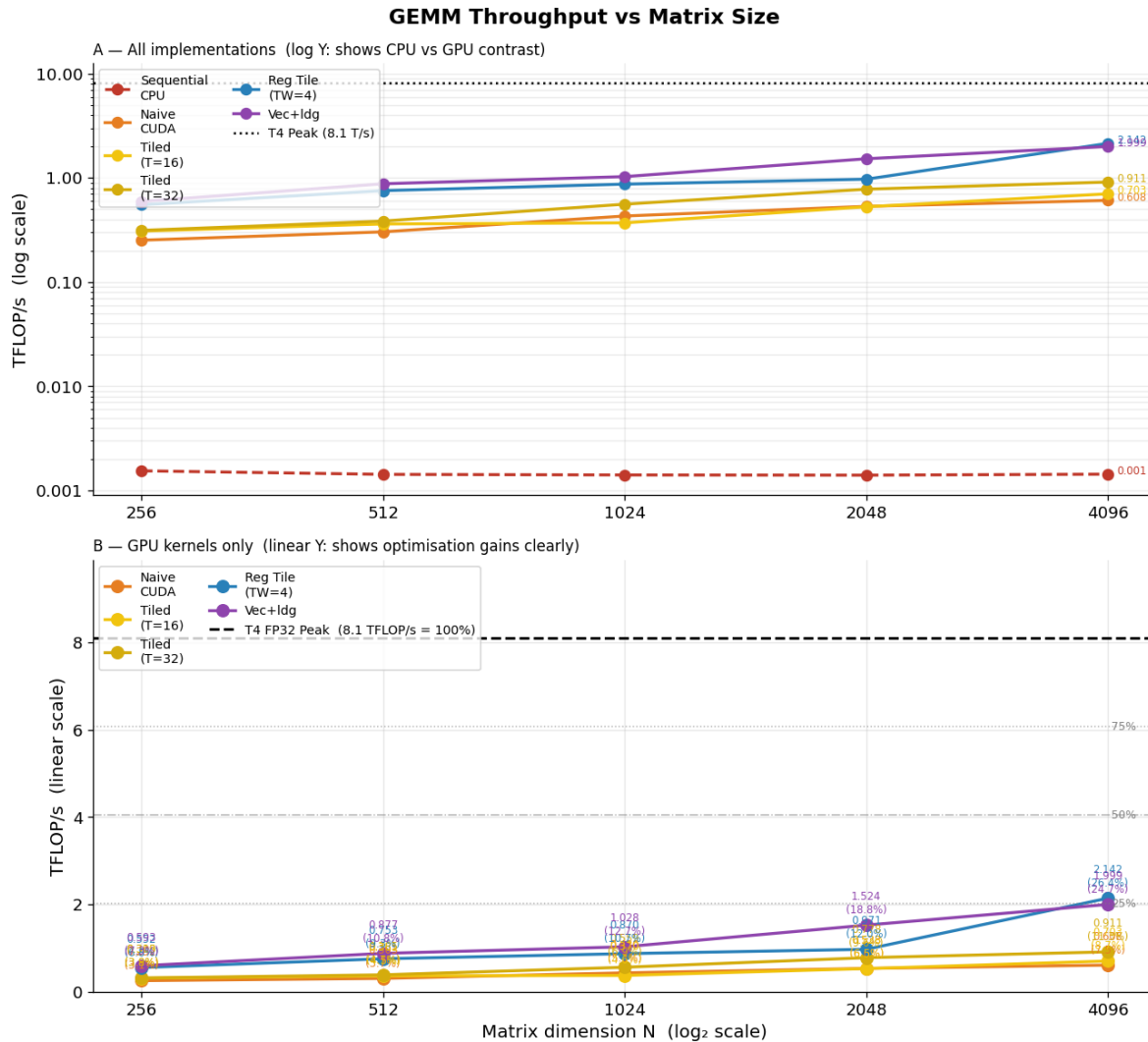


Figure 4.2: TFLOP/s versus matrix size for all implementations.

4.3 Execution time scaling

Figure 4.3 presents the corresponding execution time. The trend is the inverse of throughput: the sequential CPU version grows rapidly with N , while the GPU implementations reduce the runtime substantially. The benefit becomes especially visible for $N = 2048$ and $N = 4096$, where the optimized kernels can amortize their setup and synchronization costs over a much larger amount of computation.

This figure is important because it shows that throughput improvements are not just numerical artifacts: they translate directly into lower execution time, which is the metric that matters most for end-to-end performance.

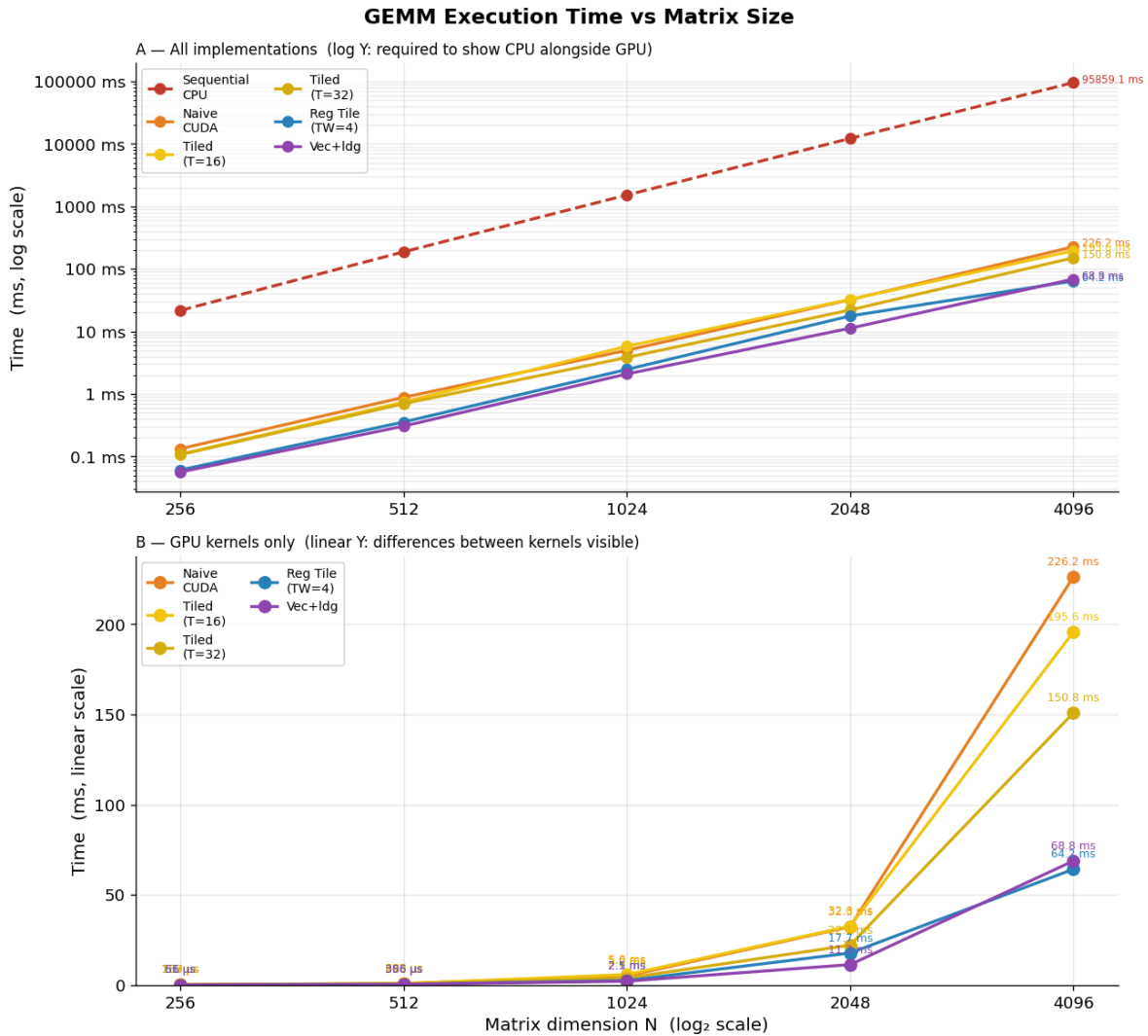


Figure 4.3: Execution time versus matrix size for all implementations.

4.4 Speedup relative to the naive kernel

Figure 4.4 isolates the effect of each optimization by normalizing throughput against the naive GPU baseline at $N = 1024$. This makes it easier to interpret the contribution of each step. The naive kernel is the reference point, the tiled kernel with $TILE = 16$ does not improve much, $TILE = 32$ provides a clear gain, register tiling gives another significant step forward, and vectorized loads with `__ldg()` provide the highest speedup.

The ranking is consistent with the expected theory: optimizations that increase data reuse and reduce traffic to global memory produce the largest benefits, while changes that only affect low-level memory layout or instruction delivery give smaller gains.

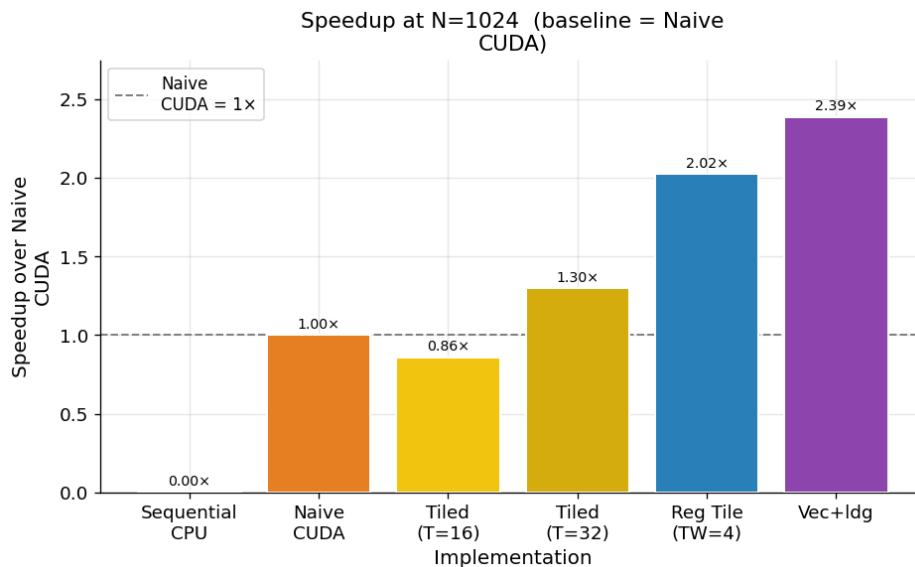


Figure 4.4: Speedup of each implementation relative to the naive GPU kernel at $N = 1024$.

4.5 Arithmetic intensity at $N = 1024$

Figure 4.5 shows the arithmetic intensity of each implementation at $N = 1024$. The most important observation is the jump from the naive kernel to the tiled kernels: tiling increases the amount of computation performed per byte fetched from global memory by reusing each loaded tile multiple times in shared memory. Register tiling increases the effective reuse even further by keeping partial results and temporary values in registers.

The vectorized `__ldg()` version does not materially change arithmetic intensity, which is expected: it improves how data is loaded, but not how much data is required. For that reason, its AI remains close to the register-tiled kernel. This distinction is important because not every performance improvement comes from changing the arithmetic intensity; some optimizations reduce instruction overhead or improve the efficiency of the memory path without changing the FLOP/byte ratio.

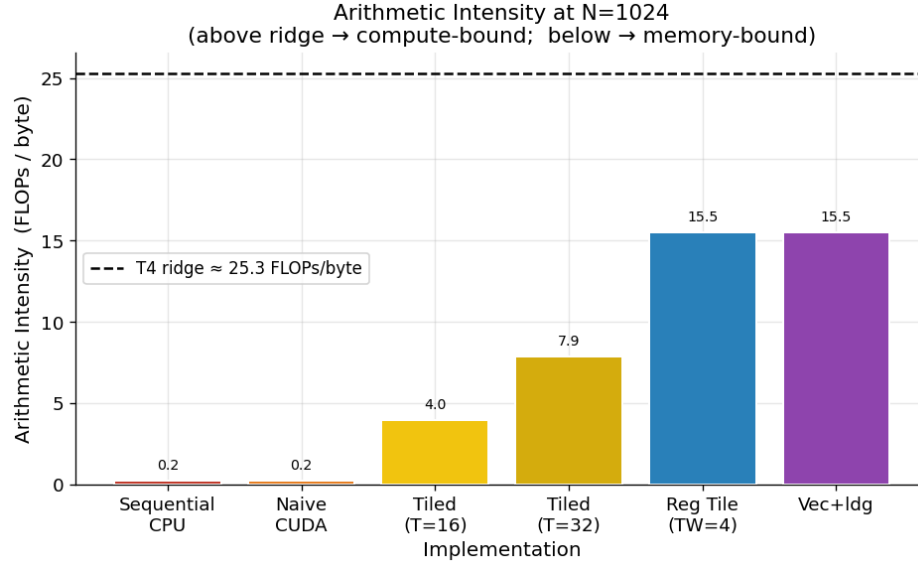


Figure 4.5: Arithmetic intensity of each implementation at $N = 1024$.

4.6 Hardware efficiency

Figure 4.6 compares each implementation against the T4 peak FP32 throughput. The naive kernel uses only a small fraction of the theoretical peak, the tiled kernels improve that fraction modestly, and the register-tiled and vectorized versions reach the highest efficiency in the project. Even so, none of the implementations comes close to saturating the hardware peak, which shows that the kernel is still limited by a combination of memory behavior, synchronization cost, and instruction overhead.

The best efficiency is achieved by the vectorized `__ldg()` kernel, but the gap to peak remains large. This is a common result in hand-written GEMM kernels: reaching peak performance usually requires more aggressive techniques such as specialized instruction paths and deeper architecture-specific tuning.

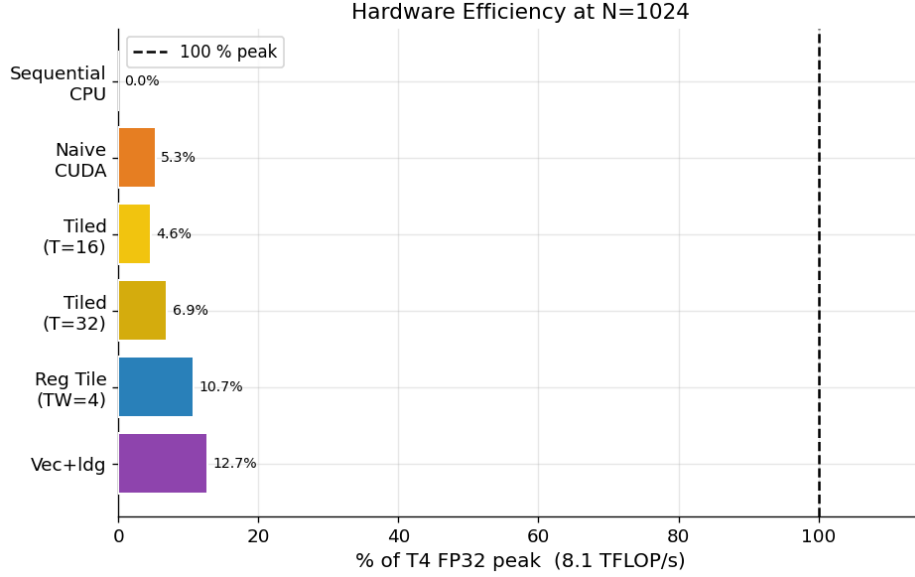


Figure 4.6: Hardware efficiency of each implementation relative to the T4 FP32 peak.

4.7 Roofline interpretation

The roofline plot in Figure 4.7 provides the most compact explanation of the whole experiment. Low-arithmetic-intensity kernels, such as the sequential and naive versions, lie in the bandwidth-limited regime. As arithmetic intensity increases through tiling and register blocking, the kernels move toward the compute-limited region and achieve higher throughput. The vectorized kernel improves the delivery of data to the compute units, which helps it approach the upper part of the roofline more closely.

This plot ties together the previous figures: it explains why higher arithmetic intensity, higher throughput, and lower execution time appear together in the optimized versions.

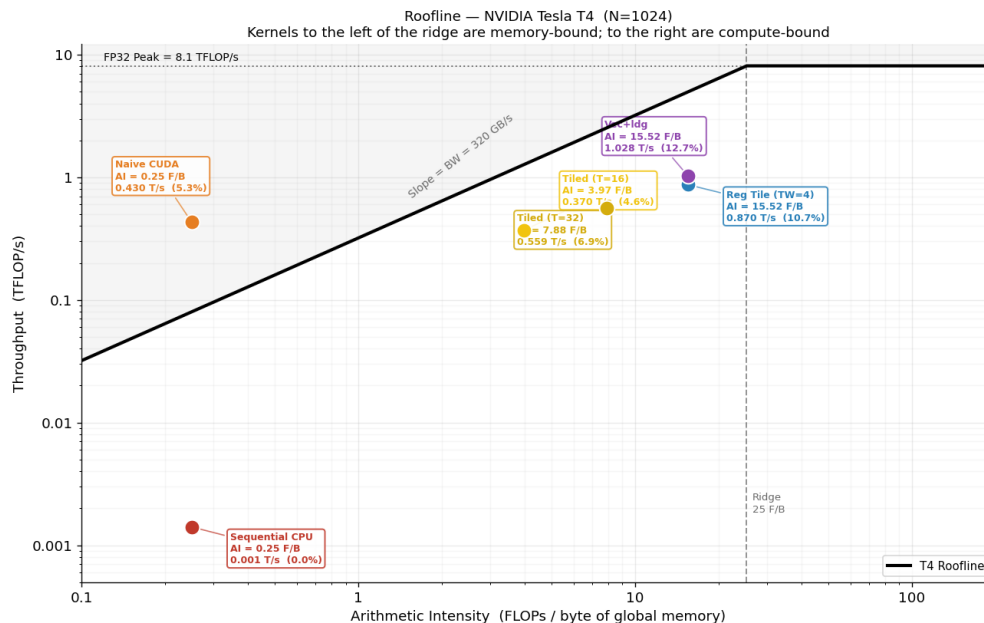


Figure 4.7: Roofline analysis at $N = 1024$.

4.8 Summary

Taken together, the figures show a consistent progression: the sequential CPU code establishes the baseline, the naive GPU kernel introduces parallelism, tiled GEMM improves reuse in shared memory, register tiling increases reuse in registers, and vectorized loads plus `__ldg()` improve the efficiency of the load path. The strongest gains come from the optimizations that reduce effective memory cost per FLOP, while the smallest gains come from changes that do not modify the real access pattern of the kernel.

Chapter 5

Conclusion

Across the full sequence of implementations, the largest improvement over the previous version came from moving from the sequential CPU baseline to the naive CUDA kernel, which reduced execution time by roughly two orders of magnitude and reached a speedup of about $305\times$ at $N = 1024$. Among the GPU-side optimizations, register tiling produced the most meaningful gain because it increased data reuse inside registers and reduced the effective cost of memory traffic. By contrast, the smallest improvement came from the bank-conflict padding step, which did not improve performance in this kernel and instead introduced a small slowdown, since the original shared-memory access pattern was already conflict-free.

The best overall performance was obtained by the register-tiled kernel, reaching 2.1424 TFLOP/s. Compared with the NVIDIA T4 FP32 peak of 8.1 TFLOP/s, this corresponds to approximately 26.4% of peak performance, leaving a gap of about 5.96 TFLOP/s. This gap indicates that the implementation is still limited by memory behavior, instruction overhead, and incomplete hardware utilization. A technique not covered in this lab that could close the gap further is the use of tensor cores through mixed-precision WMMA kernels, which can significantly raise throughput on supported hardware.

Bibliography

- [1] techpowerup. *NVIDIA Tesla T4 Specs*. URL: <https://www.techpowerup.com/gpu-specs/tesla-t4.c3316>.